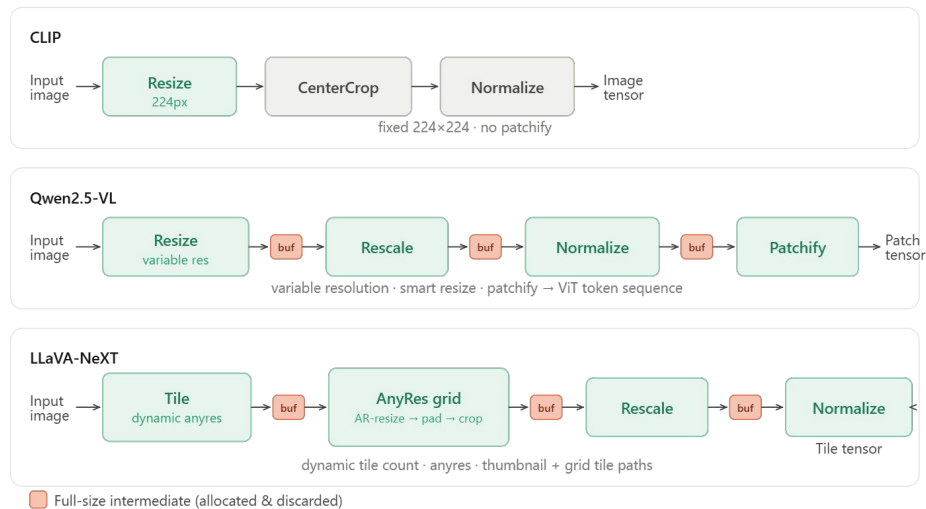


Algorithm-Schedule Separation for VLM Image Preprocessing

Julien Darve
CS 348k Final Presentation

VLMs have **Variable-Sized** Image Preprocessing

- Classic Vision Language Model (VLMs) preprocessing was simple, with fixed resolution output
- Modern VLMs have moved to **native resolution** image input
- Different models have **unique** and **complex** pre-processing pipelines
- Hugging Face implementations patch together multiple libraries (PIL, numpy torch) that create **large, wasteful intermediate memory** buffers



Solution: Separating Algorithm from Schedule

Inspired by Halide: For the same pre-processing algorithm, we explore multiple schedules

- **Naive “v1”**: Intermediate buffer per operation
- **Pointwise “v2”**: Fusing pointwise operations (*compute_at*)
- **Fused “v3”**: Fused end to end (*store_at*, *write_via*)
 - Scatter (loop over input pixels) instead of gather (loop over output patches)
- **Parallelism**: Multi-threaded (*parallel*)
- Scheduling choices are classified and dispatched into specialized Numba kernels

Algorithm — defined once

```
# qwen.py
pipeline = (
    Pipeline()
    .add(Func("resize", []))
    .add(Func("rescale",
["resize"]))
    .add(Func("normalize",
["rescale"]))
    .add(Func("patchify",
["normalize"]))
)
```

Schedule — swapped independently

```
v1 · naive
Schedule(
    "resize": StageSchedule( parallel="batch" ),
    "rescale": StageSchedule(),
    "normalize": StageSchedule(),
    "patchify": StageSchedule(),
)

v2 · pointwise fused
Schedule(
    "resize": StageSchedule( parallel="batch" ),
    "rescale": StageSchedule( compute_at="inline" ),
    "normalize": StageSchedule( compute_at="inline" ),
    "patchify": StageSchedule(),
)

v3 · fully fused
Schedule(
    "resize": StageSchedule( parallel="batch" ),
    "rescale": StageSchedule( compute_at="inline" ),
    "normalize": StageSchedule( compute_at="inline" ),
    "patchify": StageSchedule( compute_at="inline" ,
                                store_at="root",
                                write_via="qwen_patch_coords"
                                ),
    preallocate_output=True,
)
```

!hm. Different schedule. Different cost.

Is VLM pre-processing the right application for separating schedule from algorithm?

Scheduling

Tuning the scheduling with a fixed algorithm changes performance

Expressiveness

The results generalize to multiple models (Qwen, LLaVA-NeXT)

Performance

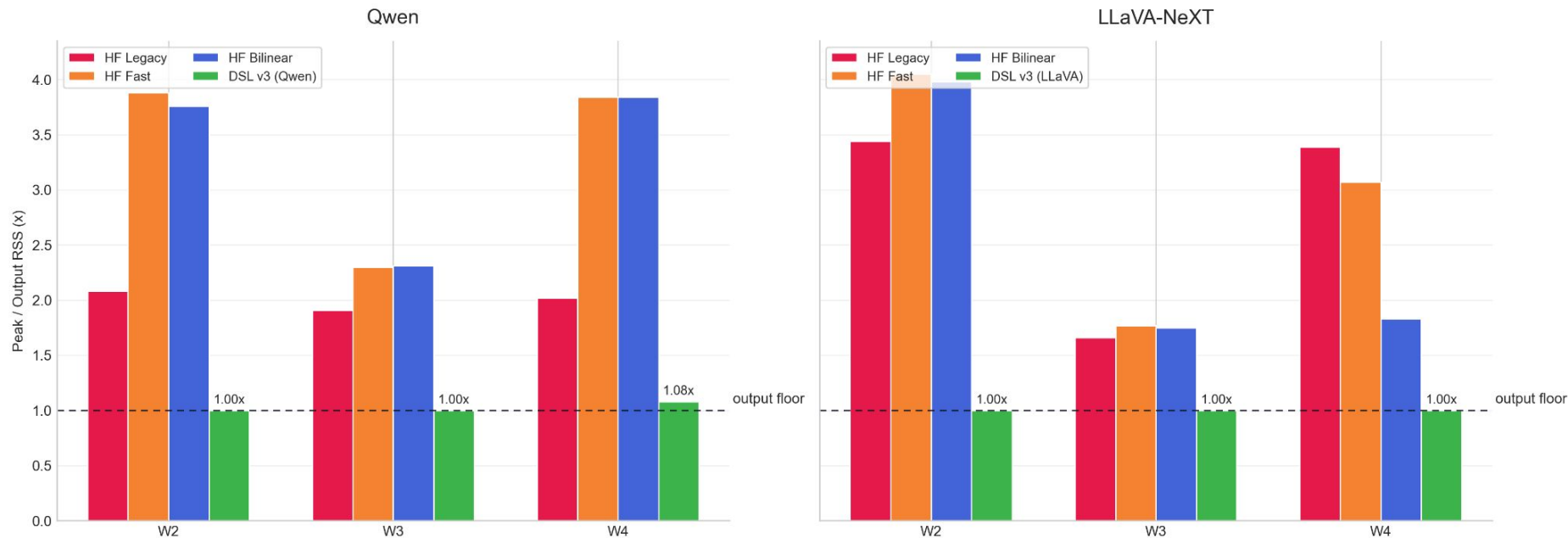
Our implementation can match or beat Hugging Face in peak memory allocation and runtime

Experimental Setup

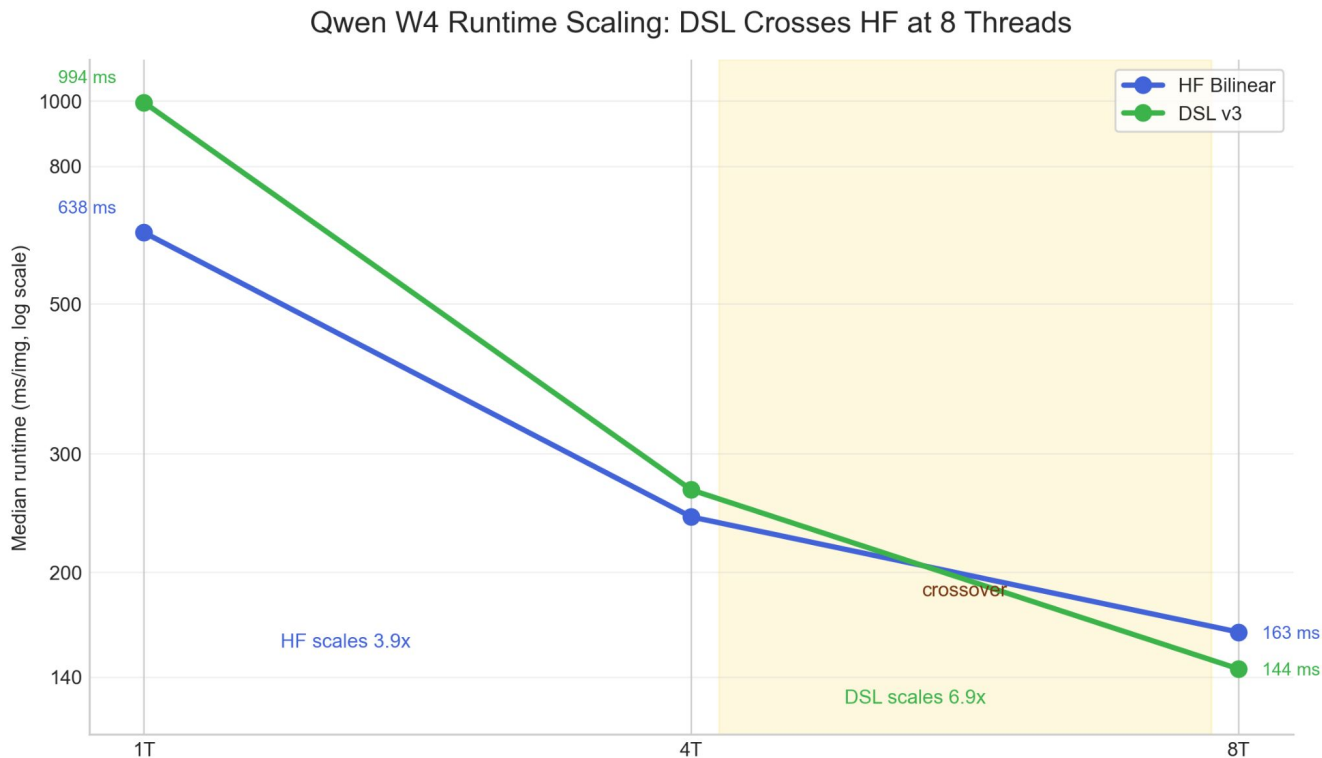
- **Baselines:**
 - HF Legacy (Numpy/PIL)
 - HF Fast (Torchvision/PIL)
 - **HF bilinear** (HF Fast with bilinear kernel)
- **Metrics:** ms/image, peak/output RSS (Resident Set Size)
- **Workloads:**
 - **Standard:** W2 (32x1024x1024)
 - **Varied:** W3 (random dimensions 256-1024, 32x mixed batch)
 - **Large:** W4 (A4 @ 300 dpi → 8x2480x3508)
- **Compute:** AWS c7i.4xlarge, 8 CPUs (1 thread / CPU)
- **Models:** Qwen2.5-VL, LLaVA-NeXT
- **Correctness:** DSL matches HF up to ~1 raw pixel level

Storage Fusion Drives Peak Memory to the Output Floor

1 Thread Peak Memory Reaches the Output Floor

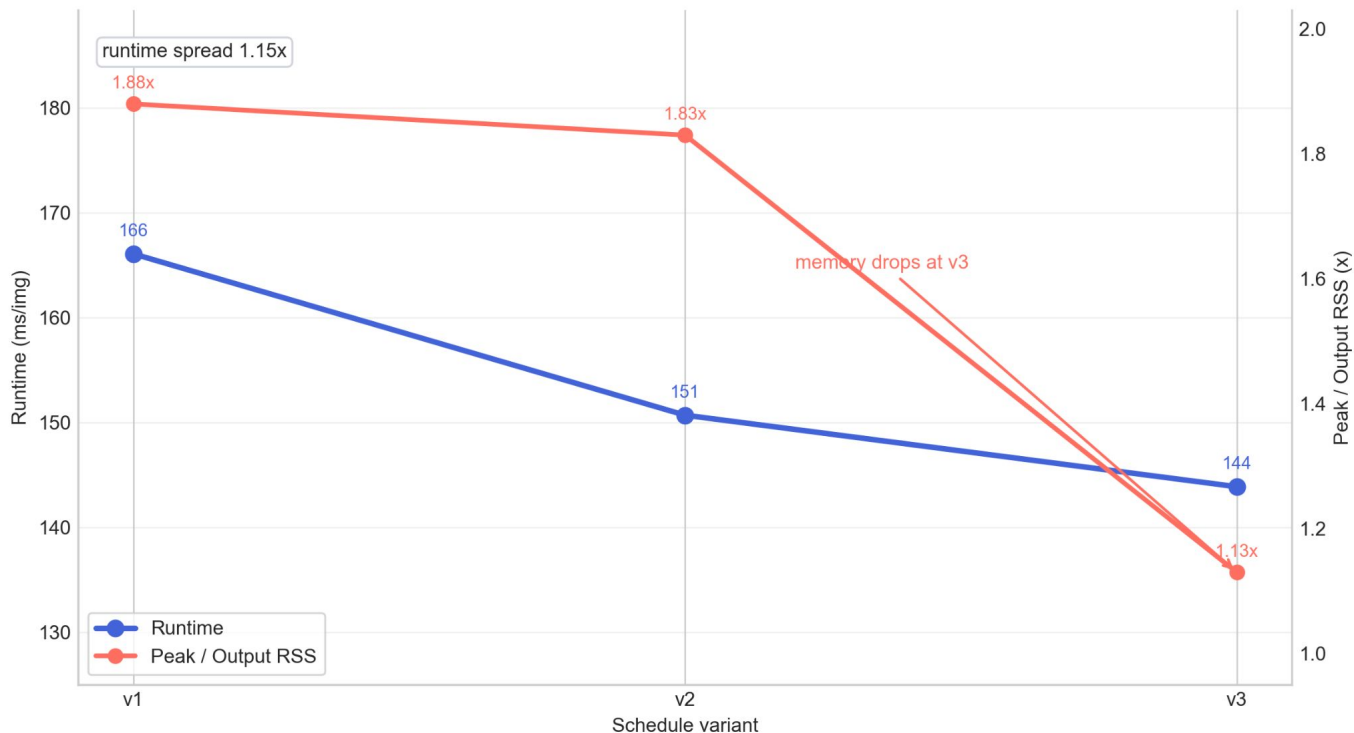


Parallelism Schedule Choice Optimizes Runtime



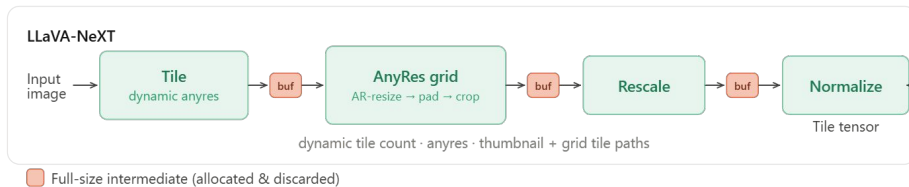
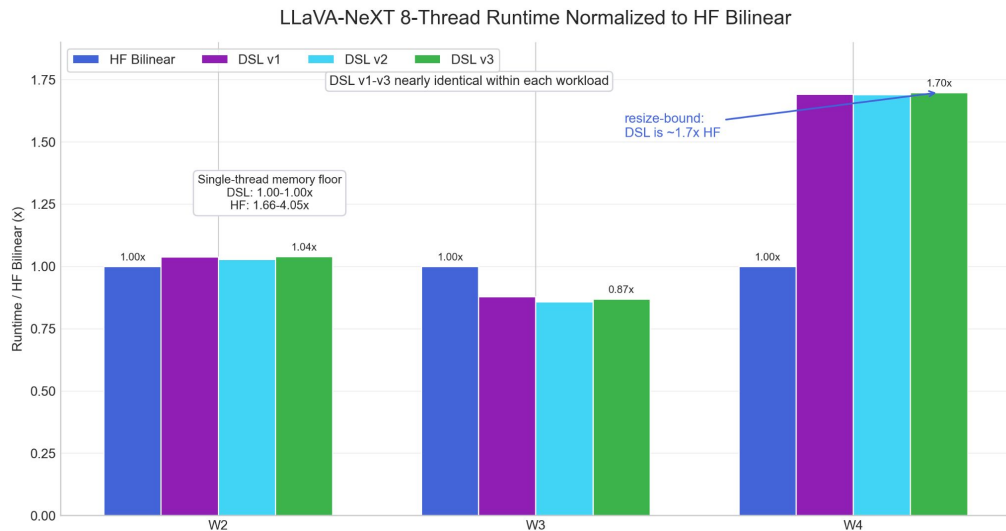
Fusion, Scatter Schedule Choices Optimize Memory

Qwen W4 Fusion Axis: Runtime Moves Little, Memory Drops at v3



Where Scheduling Doesn't Help: LLaVA is Compute-Bound

- No clear runtime gains for LLaVA
- In fact, significantly worse on W4 (large images)
- LLaVA preprocessing is dominated by a compute-intensive interpolation resize



Where our Method Succeeds

Scheduling

Tuning the schedule explores a pareto frontier

We can optimize memory (fusion) or runtime (parallelism)

Expressiveness

Qwen has clear improvements with scheduling

LlaVA did not show a significant difference

Performance

We significantly outperform HF in memory

We can't always beat HF on runtime

Conclusion / Future Work

- Existing implementations leave significant memory performance on the table
- A Halide-style algorithm/schedule split can work for VLM preprocessing, but the main payoff is memory
- The set of template kernels can be expanded into a full Halide-style DSL with compilation and code generation from primitives
- We can explore optimized kernels for bicubic / bilinear interpolation to match the HF performance gap on LLaVA, and finer-grained parallelism
- As VLM pre-processing becomes more complex and a bigger part of the compute cost, we need to remember to optimize the CPU pipeline, not just the ML model on the GPU.

Thank You!